Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Proyecto del curso · Catedrático: René Ornélyz

ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

Reporte técnico

Integrantes

Carlos Abraham Méndez Asencio · Carné 9490-24-17370

Maycol Alexander Ortiz Flores · Carné 9490-24-27213

Guatemala, noviembre de 2026

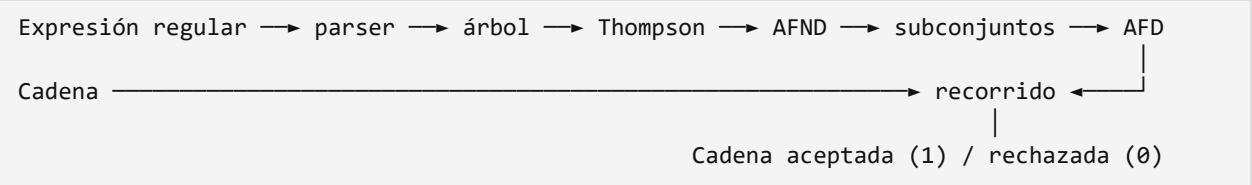
1. Objetivo

ER-AFD es un programa de línea de comando que recibe una expresión regular (ER) y una cadena. Convierte la ER en un autómata finito no determinista (AFND) con el algoritmo de Thompson, convierte ese AFND en un autómata finito determinista (AFD) con la construcción de subconjuntos y recorre la matriz de transiciones del AFD para decidir si la cadena pertenece al lenguaje. El programa devuelve `1` si la cadena se acepta y `0` si se rechaza.

Todo el proceso está escrito con estructuras estándar de Python (diccionarios, conjuntos, tuplas y colas). No se usa el módulo `re` ni ninguna otra librería de expresiones regulares.

Organización del código

Archivo	Responsabilidad
`er_afd/parser.py`	Analizador descendente recursivo. Valida la ER y construye el árbol de sintaxis.
`er_afd/automata.py`	Construcción de Thompson, cerradura ε, construcción de subconjuntos, simulación del AFD y formato de las tablas.
`er_afd/cli.py`	Comando `er-afd <ER> <cadena>`: lee los argumentos, imprime las tablas y el recorrido y devuelve el código de retorno.
`er_afd/lexer.py`	Comando adicional `er-lex`: tokeniza un texto usando un conjunto de ER.
`tests/`	Pruebas unitarias y cálculo de métricas de precisión.



2. Análisis de la expresión regular

El parser es descendente recursivo: hay una función por cada nivel de precedencia. La gramática es:

```
union  := concat ( "|" concat )*
concat := repeat repeat*
repeat := atom "*"*
atom   := literal | "." | "ε" | conjunto | escape | "(" union ")"
```

Como `union` llama a `concat`, que llama a `repeat`, que llama a `atom`, la precedencia queda de mayor a menor así: paréntesis, cerradura `*`, concatenación y unión. Por ejemplo, `ab*` es una `a` seguida de cero o más `b`, y `a|bc` es `a` o `bc`. La concatenación no tiene operador: se reconoce cuando un átomo sigue a otro.

Símbolos y secuencias especiales

Entrada	Cómo se interpreta
`.`	Cualquier carácter ASCII (códigos 0 a 127). Se agrega a Σ como el símbolo `.`.
`ε`	Cadena vacía. Produce una transición ε en el AFND.
`[abc]`, `[a-z]`	Se expanden a cada carácter del conjunto; `[abc]` equivale a `a|b|c`.
`[^abc]`	Un solo símbolo de Σ que representa todos los caracteres ASCII excepto los del conjunto.
`\N`, `\A`, `\a`, `\z`	Dígitos, mayúsculas, minúsculas y todas las letras, igual que `[0-9]`, `[A-Z]`, `[a-z]` y `[a-zA-Z]`.
`\hXX`	El carácter con código hexadecimal `XX` (de 00 a 7F). `\h20` es el espacio.
`\c`	Cualquier otro carácter después de `\` se toma literal: `\*`, `\\|`, `\\`, `\p`.

El alfabeto Σ se arma en el orden en que los símbolos aparecen en la ER. Si la ER contiene `ε`, ocupa su posición; si no, se agrega al final del alfabeto del AFND, como pide el enunciado. Una ER vacía, un paréntesis sin cerrar, un rango invertido como `[z-a]` o un `\h` sin dos dígitos son errores de sintaxis: el programa indica la posición del error y devuelve `2`.

3. Construcción de Thompson (ER → AFND)

El árbol se recorre en postorden. Cada nodo produce un fragmento con un estado de entrada y uno de salida, y los fragmentos se combinan según la operación:

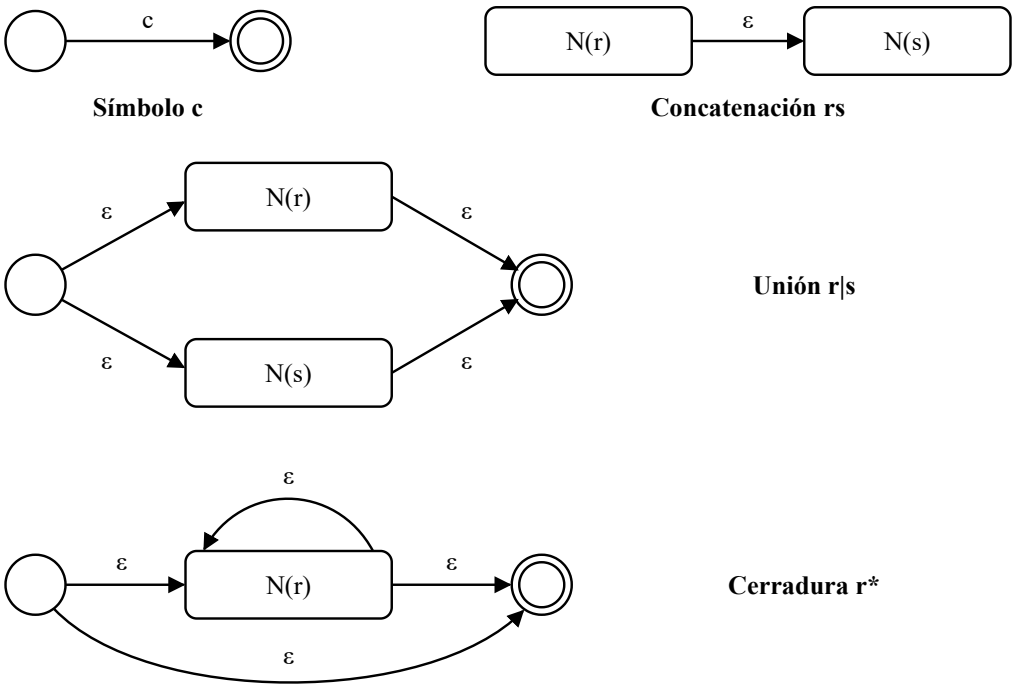


Figura 1. Fragmentos de Thompson. El círculo doble es el estado de salida del fragmento.

Nodo	**Regla en `thompson()`**
Símbolo o conjunto	Dos estados nuevos y una transición con la etiqueta del símbolo.
`ε`	Dos estados nuevos unidos por una transición ε.
Concatenación `rs`	Transición ε de la salida de `r` a la entrada de `s`.
Unión `r|s`	Estado inicial nuevo con ε hacia cada rama y estado final nuevo que recibe ε desde cada rama.
Cerradura `r*`	Estado inicial y final nuevos; ε para entrar a `r`, para repetirla, para salir y para saltarla.

El recorrido usa una pila en lugar de recursión, así que expresiones largas como `aaaa…a` no llegan al límite de recursión de Python. Al terminar, los estados se renumeran para que el estado inicial sea `q0` (la unión y la cerradura crean su estado inicial después de sus hijos). El resultado tiene un solo estado final.

Cerradura ε

La cerradura ε de un conjunto de estados son todos los estados alcanzables sin consumir símbolos. Se calcula con una búsqueda en profundidad y un conjunto de visitados, para que los ciclos ε de la cerradura de Kleene no se recorran infinitamente:

```
cerradura(S):
    visitados = S;  pendientes = S
    mientras haya pendientes:
        q = sacar(pendientes)
        para cada t con q --ε--> t:
            si t no está en visitados: agregar t a visitados y a pendientes
    devolver visitados
```

4. Construcción de subconjuntos (AFND → AFD)

Cada estado del AFD es un conjunto de estados del AFND. `Q0` es la cerradura ε de `{q0}`. Una cola procesa los subconjuntos nuevos y les asigna nombres `Q1`, `Q2`, … en el orden en que aparecen:

```
Q0 = cerradura({q0});  cola = [Q0]
mientras la cola no esté vacía:
    S = sacar(cola)
    para cada símbolo c:
        T = cerradura(mover(S, c))
        si T está vacío: no hay transición
        si T es nuevo: darle el siguiente nombre Qi y meterlo a la cola
        δ(S, c) = T
S es final si contiene el estado final del AFND
```

Cuando `mover(S, c)` no tiene destinos no se agrega ninguna transición. No se crea un estado pozo: al recorrer la cadena, si un símbolo no tiene transición la cadena se rechaza en ese momento. Así la tabla muestra solo las transiciones útiles, igual que el ejemplo del enunciado.

Símbolos que se solapan

En una ER como `a|.` la letra `a` coincide con el símbolo `a` y también con el punto. Si el AFD usara las etiquetas tal cual, desde un mismo estado habría dos transiciones posibles para la letra `a` y dejaría de ser determinista. Por eso, antes de la construcción, `_partitions()` agrupa los 128 caracteres ASCII según las etiquetas que los aceptan:

Caracteres	Etiquetas que los aceptan	Columna del AFD
`a`	`a` y `.`	`a`
cualquier otro	solo `.`	`.` (cualquier carácter distinto de `a`)

Con `a|.` el AFD queda con `Q0|a|Q1` y `Q0|.|Q2`. Para `a` se usa la fila `a` y para cualquier otro carácter la fila `.`. Solo si dos conjuntos amplios se solapan entre sí, como `.` y `[^a]`, aparece una columna auxiliar `C0`, y el programa imprime qué caracteres contiene. Con `--verbose` se muestran todas las clases y el subconjunto del AFND de cada estado.

5. Ejemplo completo: (a|b)*abb

Es el autómata más grande de los casos de prueba y combina los tres operadores. Ejecución: `er-afd '(a|b)*abb' 'aabb'`.

AFND de Thompson

Q = {q0, …, q13}, Σ = {a, b, ε}, F = {q13}. `q0` y `q7` son la entrada y la salida de la cerradura; `q5` y `q6` las de la unión `a|b`; `q1→q2` y `q3→q4` leen `a` y `b` dentro de la unión, y `q8–q13` forman la concatenación `abb`, unida con transiciones ε.

Subconjuntos

AFD	Subconjunto del AFND	con a	con b	Final
Q0	{q0,q1,q3,q5,q7,q8}	Q1	Q2	
Q1	{q1,q2,q3,q5,q6,q7,q8,q9,q10}	Q1	Q3	
Q2	{q1,q3,q4,q5,q6,q7,q8}	Q1	Q2	
Q3	{q1,q3,q4,q5,q6,q7,q8,q11,q12}	Q1	Q4	
Q4	{q1,q3,q4,q5,q6,q7,q8,q13}	Q1	Q2	sí

`Q4` es final porque contiene `q13`. Cada estado del AFD "recuerda" cuánto del sufijo `abb` se ha leído: `Q1` terminó en `a`, `Q3` en `ab` y `Q4` en `abb`.

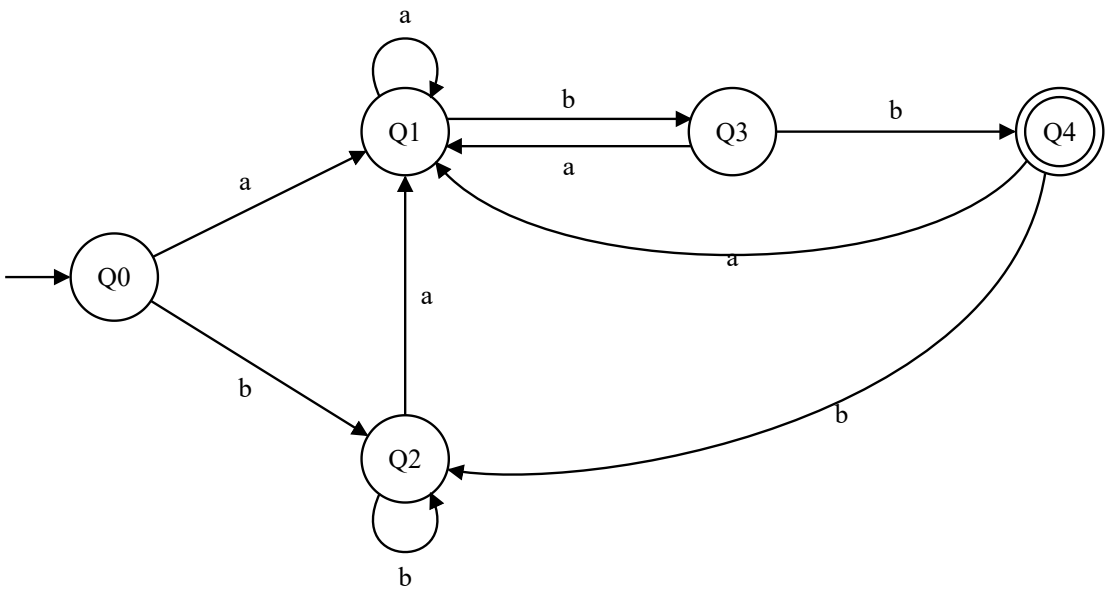


Figura 2. AFD generado para (a|b)*abb. Q0 es el inicial y Q4 el de aceptación.

Recorrido de aabb

```
Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

El recorrido consulta la tabla una vez por carácter: `estado = δ(estado, c)`. Termina en `Q4`, que es final, así que la cadena se acepta y el programa devuelve `1`. Con `aaba` el recorrido termina en `Q1`, que no es final, y devuelve `0`.

6. Simulación y códigos de retorno

Situación	Salida	Retorno
Termina en un estado final	`Cadena aceptada`	1
Termina en un estado no final, o un símbolo no tiene transición	`Cadena rechazada`	0
ER inválida, carácter no ASCII o archivo inexistente	`Error: …` (stderr)	2

Que una aceptación devuelva `1` lo pide el enunciado, aunque en Linux normalmente `0` indica éxito. El código `2` separa los errores de los rechazos, para que una ER mal escrita no se confunda con una cadena rechazada.

Complejidad

- Thompson crea a lo sumo dos estados por símbolo u operador: el AFND es lineal en el tamaño de la ER.
- La construcción de subconjuntos puede generar hasta 2^n estados en el peor caso. El límite por defecto es de 4096 estados de AFD y se cambia con `--max-states`.
- El recorrido es lineal en la longitud de la cadena: una consulta a la tabla por carácter.

7. Funciones adicionales

- `--verbose`: muestra el subconjunto del AFND de cada estado del AFD.
- `--json`: los dos autómatas y el recorrido en JSON.
- `--dot PREFIJO`: genera archivos de Graphviz para dibujar el AFND y el AFD.
- `--file RUTA`: lee la cadena desde un archivo.
- `er-lex`: usa un conjunto de ER (en JSON) para separar un archivo fuente en tokens. Se queda con la coincidencia más larga y, si hay empate, con la primera regla.

8. Referencias

1. Ornélyz, R. (2026). *ER-AFD: Convertidor de ER a AFD. Proyecto del curso.* Universidad Mariano Gálvez de Guatemala.
2. Thompson, K. (1968). Regular expression search algorithm. *Communications of the ACM, 11*(6), 419-422.
3. Rabin, M. O. y Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development, 3*(2), 114-125.
4. Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.ª ed.), sección 3.7. Pearson.

Usamos Codex como asistente durante el desarrollo; el detalle está en `FUENTES.md`.